西南交通大學
Southwest Jiaotong University

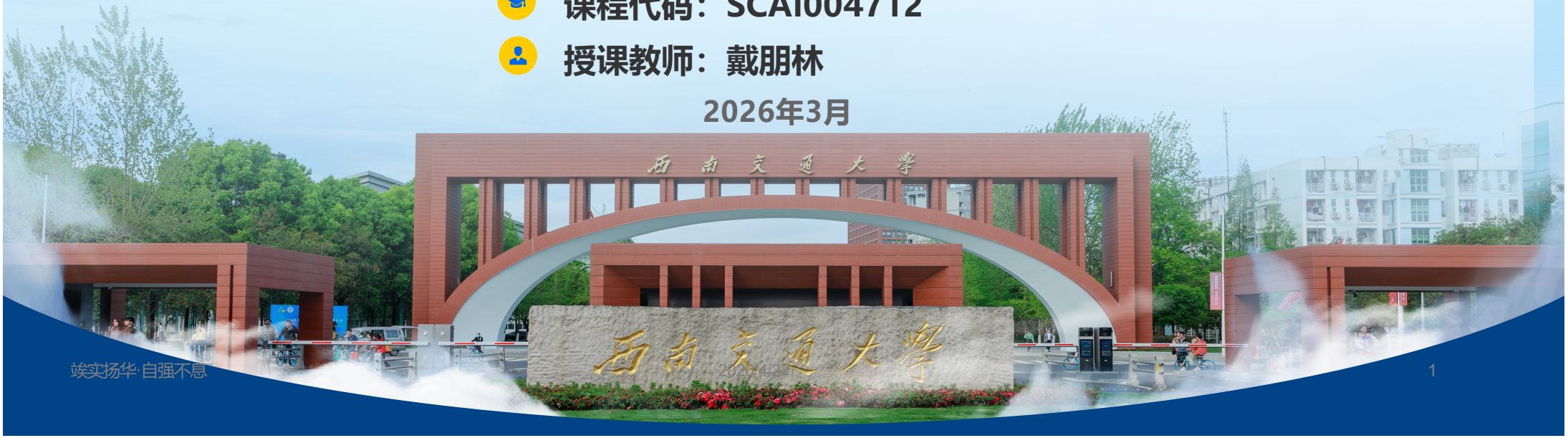
云计算技术

第8讲 分布式文件系统

课程代码：SCAI004712

授课教师：戴朋林

2026年3月



01 本讲内容大纲

西南交通大学 Southwest Jiaotong University

01 存储类型概述

块 vs 文件 vs 对象

02 Google File System

设计目标 / 架构 / 读写流程

03 HDFS 架构与实践

NameNode / 副本策略 / HA / Federation

04 对象存储

Amazon S3 / MinIO

05 Ceph 统一存储

CRUSH 算法 / 三大接口

06 对比与选型

全景对比 / 云厂商矩阵

07 应用案例与代码剖析

数据湖 / 高校 Ceph 私有云

08 思考题 & Q&A

深度问题 / 课后实践

1 存储类型概述

- 三类存储概念
- 性能对比
- 选型建议

01 三种存储：块存储、文件存储、对象存储

西南交通大学 Southwest Jiaotong University

块存储（Block Storage）

将存储空间划分为固定大小的块（Block），提供裸设备级别的访问接口。

类比：一块未格式化的本地硬盘

访问方式：设备路径（如 Linux下的 /dev/sda）

代表：AWS EBS、Ceph RBD

文件存储（File Storage）

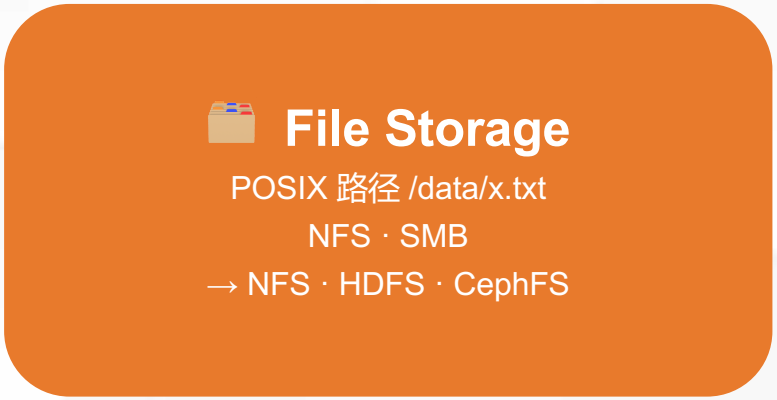
提供文件和目录的层级树状结构，通过文件路径访问数据。

类比：一个网络共享文件夹

访问方式：符合POSIX的文件路径（如 /data/file.txt）

代表：NFS、HDFS、CephFS





01 三种存储类型

西南交通大学 Southwest Jiaotong University

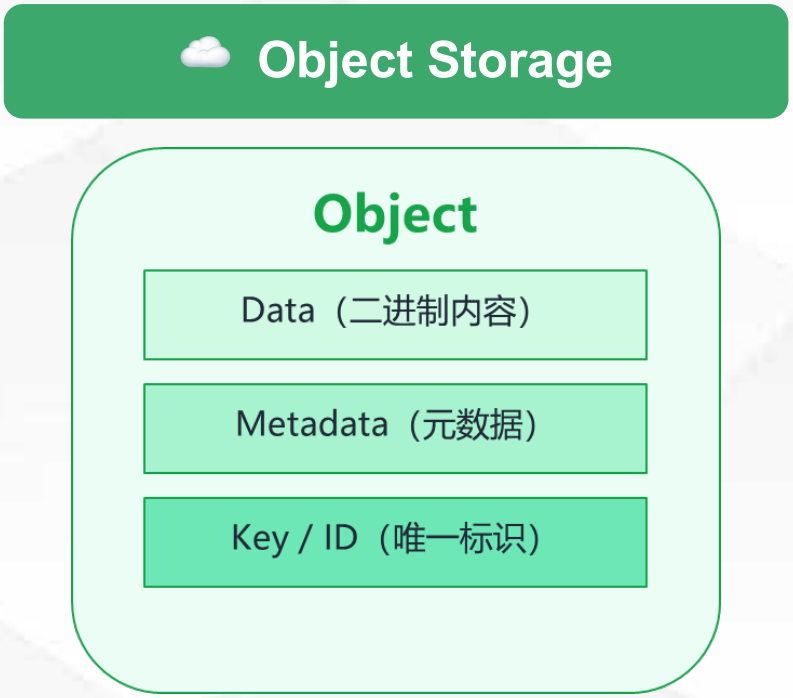
对象存储（Object Storage）

将数据作为对象（Object）存储，采用扁平命名空间。包含数据、元数据和唯一键。

类比：一个配有代客泊车凭证的无限大仓库

访问方式：HTTP RESTful API（如 GET /bucket/key）

代表：AWS S3、MinIO、阿里云OSS



02 三种存储类型对比

西南交通大学 Southwest Jiaotong University

维度	块存储	文件存储	对象存储
抽象层级	最低（裸盘）	中（树状目录）	最高（扁平桶）
访问接口	SCSI/iSCSI	POSIX/NFS/SMB	HTTP REST API
性能表现	极低延迟、高 IOPS	较高吞吐、中等延迟	侧重超高吞吐、延迟较高
扩展能力	较弱（受限于 SAN 架构）	中等（受限于目录树元数据）	极强（轻松扩展至 EB 级）
修改方式	支持随机修改	支持随机修改	只能整体覆盖替换
适用场景	数据库、虚拟机系统盘	多机共享日志、办公文档	海量图片 / 视频、冷数据归档

02 三种存储类型对比

跑系统与数据库选**块存储**，多机共享读写选**文件存储**，海量多媒体与静态资源选**对象存储**。



2 Google File System

- GFS 设计背景
- GFS 架构
- GFS 追加写入流程

01 2003年 Google面临的存储挑战

西南交通大学 Southwest Jiaotong University

- ✓ 搜索引擎需要存储PB级网页索引数据
- ✓ 使用廉价商用服务器（组件故障是常态，非例外）
- ✓ 典型的负载特征：大文件为主（GB级），追加写入（Append）为主，随机修改极少
- ✓ 高吞吐率（Throughput）比低延迟更重要

PB
网页索引量级

GB
单文件平均大小

x86
廉价商用机

Append
顺序追加为主

02 GFS四大核心假设

1. **系统由大量廉价易损节点组成**（必须具备自我监控、容错、自动恢复机制）
2. **系统主要存储大文件**（数百万个100MB以上的对象，无需针对KB级小文件优化）
3. **读写模式固定**（大规模流式读取、少量随机读取；主要为顺序追加写入）
4. **并发追加语义**（多个客户端并发追加同一文件，需保证高效与一致性）



03 GFS架构：核心组件职责

西南交通大学 Southwest Jiaotong University

Master（控制流）：唯一的主节点。管理整个文件系统元数据（目录树、文件到Chunk的映射、Chunk位置）。

ChunkServer（数据流）：存储实际数据块（Chunk）。每个Chunk默认大小64MB，通常保存3个副本。

Client（客户端）：向Master询问元数据，随后直接与ChunkServer进行数据交互。



04 GFS 架构关键创新点 (1/2)

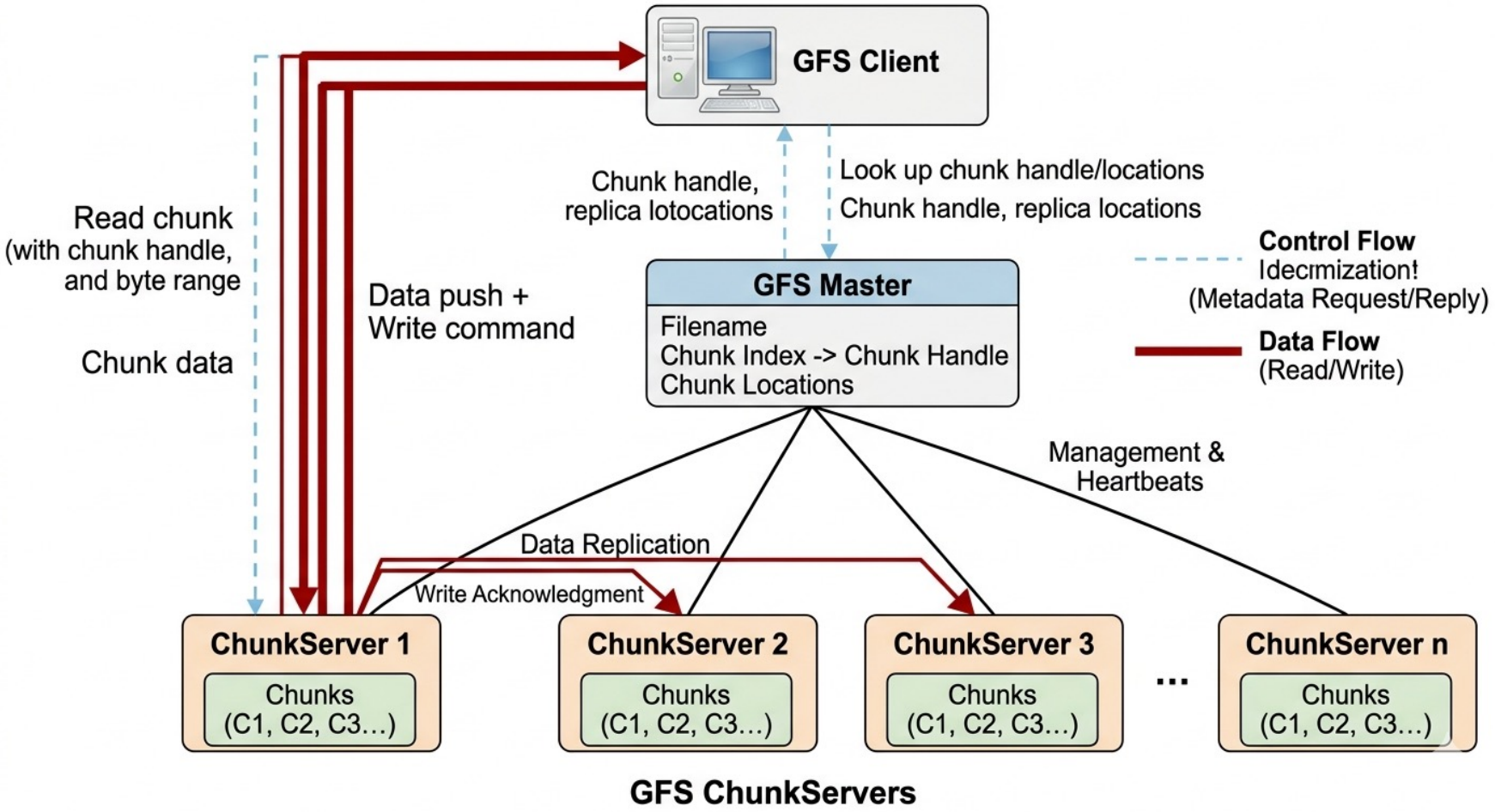
西南交通大学 Southwest Jiaotong University

1.**单一Master设计**：大幅简化设计架构，避免了复杂的分布式一致性协议（如Paxos/Raft用于元数据），但通过内存映射和操作日志解决性能瓶颈。

2. **控制流与数据流分离**：Client仅找Master问路，数据传输绝不经过Master，避免Master成为网络瓶颈。

3. **大Chunk策略（64MB）**：远大于普通操作系统的4KB，极大减少了Master的元数据管理压力

04 GFS 架构总览（控制流 vs 数据流）



05 GFS追加写入流程

西南交通大学 Southwest Jiaotong University

为什么关注追加（Record Append）？

GFS专门优化了追加写入，允许数百个客户端并发向同一文件追加，而不需加分布式锁。

传统写入：随机修改

需分布式锁 · 高冲突

高延迟 · 低吞吐

GFS 追加：Record Append

数百客户端无锁并发

极高吞吐 · 适合日志场景

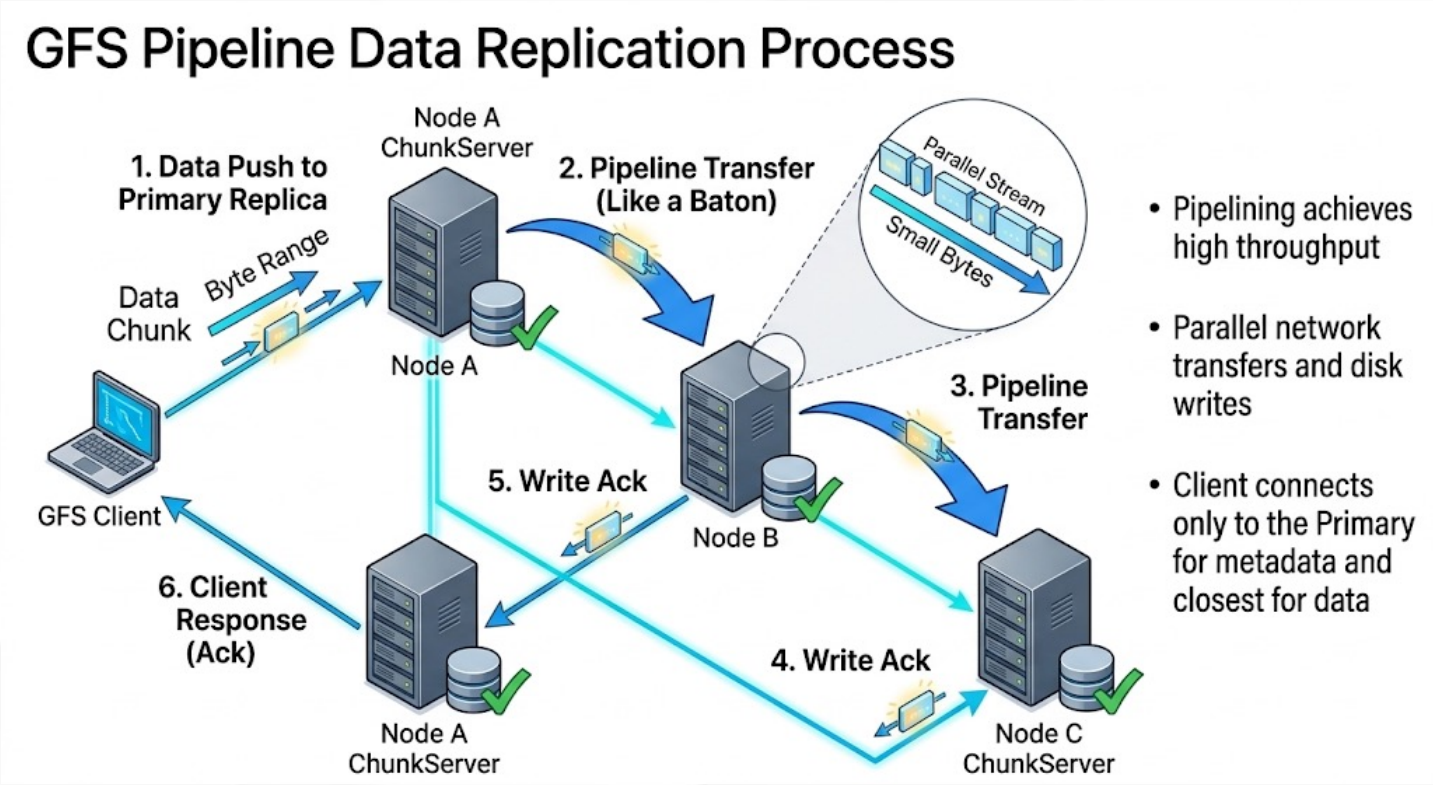
05 GFS追加写入流程

写入流程设计：流水线（Pipeline）机制

- **授权主副本**：Master在3个副本中指定一个为Primary（主），负责确定写入顺序。
- **数据链式推送**：Client不直接向3个副本同时发数据，而是发给最近的副本A，A推给B，B推给C（充分利用节点全双工带宽）。
- **执行写入顺序**：数据推送完成后，Client通知Primary执行，Primary定好顺序后通知所有Secondary副本按相同顺序落盘。

05 一致性妥协

一致性妥协：GFS 采用**至少一次语义**，GFS不保证副本字节级完全一致，可能存在重复废弃数据块并交由上层应用处理，秉持“够用就好” 的哲学，为了极高的并发吞吐，放弃了完美的强一致性。



3 HDFS

- HDFS 概述
- HDFS 架构
- HDFS 读写流程
- 高可用与演进

01 HDFS概述

西南交通大学 Southwest Jiaotong University

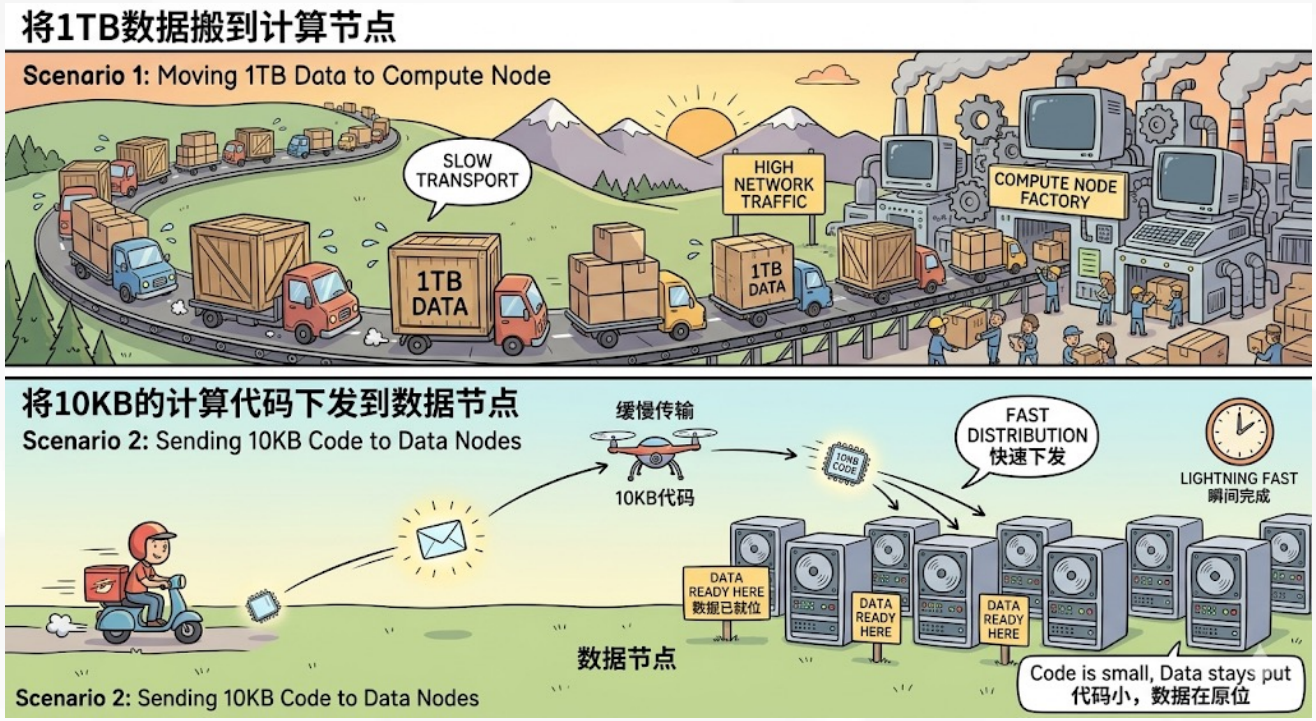
HDFS（Hadoop Distributed File System） 是Apache Hadoop大数据生态的底层基石。其设计深度借鉴了GFS论文，用Java实现。

HDFS设计的核心取舍

设计目标	放弃的特性
超大文件处理（TB/PB 级别）	不适合海量小文件存储
流式数据访问（一次写入，多次读取）	不支持低延迟数据随机访问
高容错与低成本（运行在普通硬件）	强一致性随机修改（仅支持追加）
计算向数据移动（数据本地性）	计算与存储完全分离架构

02 “计算向数据移动”的核心思想

传统架构是将数据读入内存进行计算（耗费网络带宽）；HDFS配合MapReduce/Spark，将计算程序调度到数据所在的DataNode上运行，实现网络零拷贝。



03 HDFS 架构：双心结构 (1/3)

西南交通大学 Southwest Jiaotong University

1.NameNode（大脑）

- **职责**：维护文件系统树，管理文件和目录的元数据。
- **存储介质**：全内存运行（保证查询极快），持久化依赖FsImage（命名空间镜像）和EditLog（编辑日志）。
- **瓶颈**：1GB内存大约只能管理100万个Block，因此极度惧怕海量小文件。

03 HDFS 架构：双心结构 (2/3)

西南交通大学 Southwest Jiaotong University

2. DataNode（躯干）

职责：实际存储数据的组件，以Block（默认128MB）为基本单位。

汇报机制：每3秒向NameNode发送一次心跳包（Heartbeat），每隔几个小时发送全量块报告（Block Report）。

03 HDFS 架构：双心结构 (3/3)

西南交通大学 Southwest Jiaotong University

3. Secondary NameNode（秘书，非热备）

- **注意**：它不是高可用备份！
- **作用**仅为定期将EditLog与FsImage在本地进行合并（Checkpoint），防止日志无限膨胀，减轻NameNode启动时的压力。

⚠️ 注意：Secondary NameNode ≠ 热备份

它无法在 Active NN 宕机时直接接管

📋 Checkpoint 流程

定期合并 EditLog + FsImage

防止日志无限膨胀

🎯 真正的热备

HDFS HA 中的 Standby NN

见 Slide 29

03 组件职责

西南交通大学 Southwest Jiaotong University

组件	职责
NameNode	管理文件系统命名空间和元数据；存储文件→Block 映射
DataNode	存储实际数据 Block；向 NameNode 发送心跳和块报告
Secondary NameNode	定期合并 EditLog 和 FsImage（不是热备份！）
Block	数据块，默认 128MB（Hadoop 2.x+），每个 Block 3 副本

04 Block与机架感知

西南交通大学 Southwest Jiaotong University

默认大Block（128MB起步）

为什么HDFS采用远大于操作系统的块大小?

- ✓ **最小化寻道时间开销**：如果寻道时间是10ms，传输速率100MB/s。为了让寻道时间占比降到1%以下，块至少要是100MB。
- ✓ **减少元数据**：极大减轻了NameNode的内存负担。

4KB
OS 块大小
寻道占比 99%+



128 MB
HDFS 块大小
寻道占比 < 1%

- 寻道时间 10ms
- 传输速率 100MB/s
- 块越大，寻道占比越低
- 同时大幅减少 NN 元数据

05 副本放置策略（Rack Awareness）

HDFS默认采用3副本机制，其放置算法是一项极佳的工程折中设计

- ✓ 副本1：放在客户端所在的DataNode（如果是集群外客户端，则随机选一个负载低的节点）。保证写入速度
- ✓ 副本2：放在不同机架的某DataNode上。—— 防止单机架断电或交换机故障导致数据全损
- ✓ 副本3：放在与副本2相同机架的另一个DataNode上。—— 避免跨机架传输过多占用核心交换机带宽



06 数据本地性（Data Locality）优先的读取

西南交通大学 Southwest Jiaotong University

客户端读取时，NameNode会返回Block所在DataNode的列表，列表按网络拓扑距离排序：

1. Node Local（同节点）：应用直接读取本地磁盘。速度最快。
2. Rack Local（同机架）：跨同机架交换机读取。速度次之。
3. Off Rack（跨机架）：跨核心交换机读取。最耗费带宽。



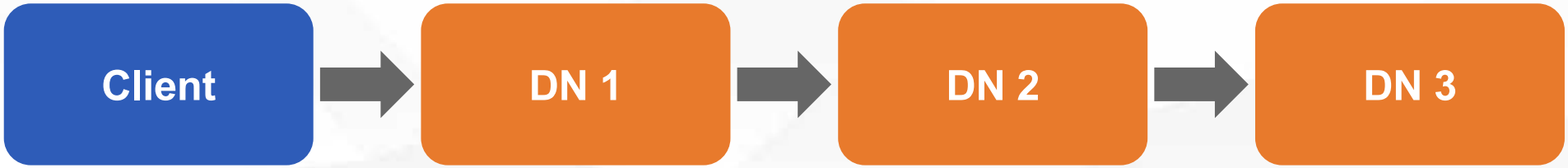
07 流水线写入与Packet传输

西南交通大学 Southwest Jiaotong University

1. 客户端并非凑齐128MB才发，而是将数据切分成多个 64KB的Packet。
2. Packet在Pipeline中以“排队流转”的方式流动：

   Client ➔ DataNode1 ➔ DataNode2 ➔ DataNode3

3. 确认机制：只要至少有一个副本写入成功，HDFS就认为写入成功，剩余副本由NameNode后续异步补充。



数据切分为 64KB Packet → 流水线推送 → 至少一个副本确认即返回成功

08 单点故障的痛点

西南交通大学 Southwest Jiaotong University

Hadoop 1.x时代，NameNode是单点故障（SPOF）。一旦宕机，整个PB级集群将彻底瘫痪。

✕ SPOF — 整集群瘫痪

NameNode 宕机 → 所有读写请求失败

PB 级数据无法访问 → 业务停摆

Hadoop 1.x 无热备机制

08 HDFS HA（高可用）架构方案

西南交通大学 Southwest Jiaotong University

采用Active/Standby双机热备架构解决单点问题：

1. Active NameNode：负责提供所有读写服务。

2. Standby NameNode：热备份，时刻准备接管。

3. JournalNode集群：取代传统的本地硬盘，用于Active和Standby之间实 时同步EditLog日志（基于Paxos变体，过半数写入成功机制）。

4. ZooKeeper与ZKFC：用于自动脑裂保护和故障切换。ZKFC监控NN心跳，一旦Active无响应，立刻通过Zookeeper发起选举，将Standby推举为新的Active。

09 Federation与EC

西南交通大学 Southwest Jiaotong University

HDFS Federation（联邦）—— 解决内存天花板

- 即使有HA，单个NameNode的内存也只能支撑几亿个文件。
- 方案：引入多个独立NameNode（类似微服务化），每个NN管理独立的Namespace（比如 /user由NN1管，/data由NN2管），底层共享同一个DataNode集群。

❌ **单 NameNode：内存饱和**

1GB 内存 ≈ 100 万 Block
亿级文件即触顶
扩展不动

✅ **Federation：多 NS 分担**

/user → NN1
/data → NN2
底层共享 DataNode 池

09 纠删码（Erasure Coding, EC）

西南交通大学 Southwest Jiaotong University

- **传统3副本的痛点**：存储开销高达200%（存1TB需要3TB硬盘），成本高昂。
- **EC技术原理**：类似于RAID 5/6。将数据分片并计算出奇偶校验块。
- **典型策略**（RS-6-3）：6份数据块 + 3份校验块 = 9块。任意丢失3块均可恢复。
- **收益**：存储开销从200%大幅下降至50%，极度适合访问量较低的冷数据（如日志归档）。

💰 **3 副本：200% 存储开销**
1TB 数据 → 3TB 硬盘

✨ **EC 编码：50% 存储开销**
1TB 数据 → 1.5TB 硬盘

4 对象存储

- 扁平命名空间
- Amazon S3
- MinIO

01 对象存储概念

对象存储 采用扁平的命名空间（无目录层级），数据以对象（Object）为单位存储在桶（Bucket）中。

```
Bucket（桶）：my-company-assets
  ├── Object: /2024/images/logo.png  (这里的路径仅仅是一个长字符串Key，并没有真实的目录)
  │      ├── Data: [图片二进制流]
  │      └── Metadata: {Content-Type: image/png, Cache-Control: max-age=3600}
```

02 核心组成要素

西南交通大学 Southwest Jiaotong University

1. **Key（对象键）**：全局唯一标识符，用户眼中的“文件路径”。
2. **Value（数据体）**：被当作不透明二进制块存储的数据，最大可达5TB。
3. **Metadata（元数据）**：与数据紧密绑定！用户可以自定义标签（如Department=Marketing），极大方便了后续的数据检索和生命周期管理。



03 Amazon S3

西南交通大学 Southwest Jiaotong University

诞生于2006年的S3（Simple Storage Service）通过极其简单的RESTful API规范，确立了云存储的行业标准规范。

S3的杀手级特性

无限扩展与11个9的持久性：底层跨多个可用区（AZ）自动复制。

存储生命周期分层（Lifecycle Management）：这是公有云对象存储最大卖点。数据可随时间推移自动降级，节约成本。

11 个 9
数据持久性
99.999999999%

2006
云存储元年
S3 发布

Lifecycle
自动分层
降本利器

03 Amazon S3存储分层降本模型

西南交通大学 Southwest Jiaotong University

- ✓ **S3 Standard（标准层）**：毫秒延迟，针对热数据，存储费贵。
- ✓ **S3 IA（低频层）**：毫秒延迟，存储便宜，但在读取时按GB收取数据取回费。
- ✓ **S3 Glacier（归档层）**：存储白菜价，但提取需要数分钟到数小时解冻。
- ✓ **Intelligent-Tiering（智能分层）**：利用机器学习自动监控访问模式进行上下层转移。



04 MinIO：什么是 MinIO

西南交通大学 Southwest Jiaotong University

诞生自2014年，由前GlusterFS创始人打造。它是一个极致轻量、超高性能且100%兼容S3 API的开源对象存储。被誉为Kubernetes时代的云原生存储利器。

~50MB
单二进制
零依赖

100%
S3 API
完全兼容

GB/s
集群吞吐
Go + SIMD

K8s
云原生
部署利器

04 MinIO 核心竞争力

西南交通大学 Southwest Jiaotong University

极简主义设计：服务端就是一个几十MB的单个二进制文件，没有任何外部依赖（不依赖数据库或缓存）。

极致性能：采用Go语言编写，深度利用SIMD指令优化底层的纠删码和哈希计算。集群规模下读写可达百GB/s。

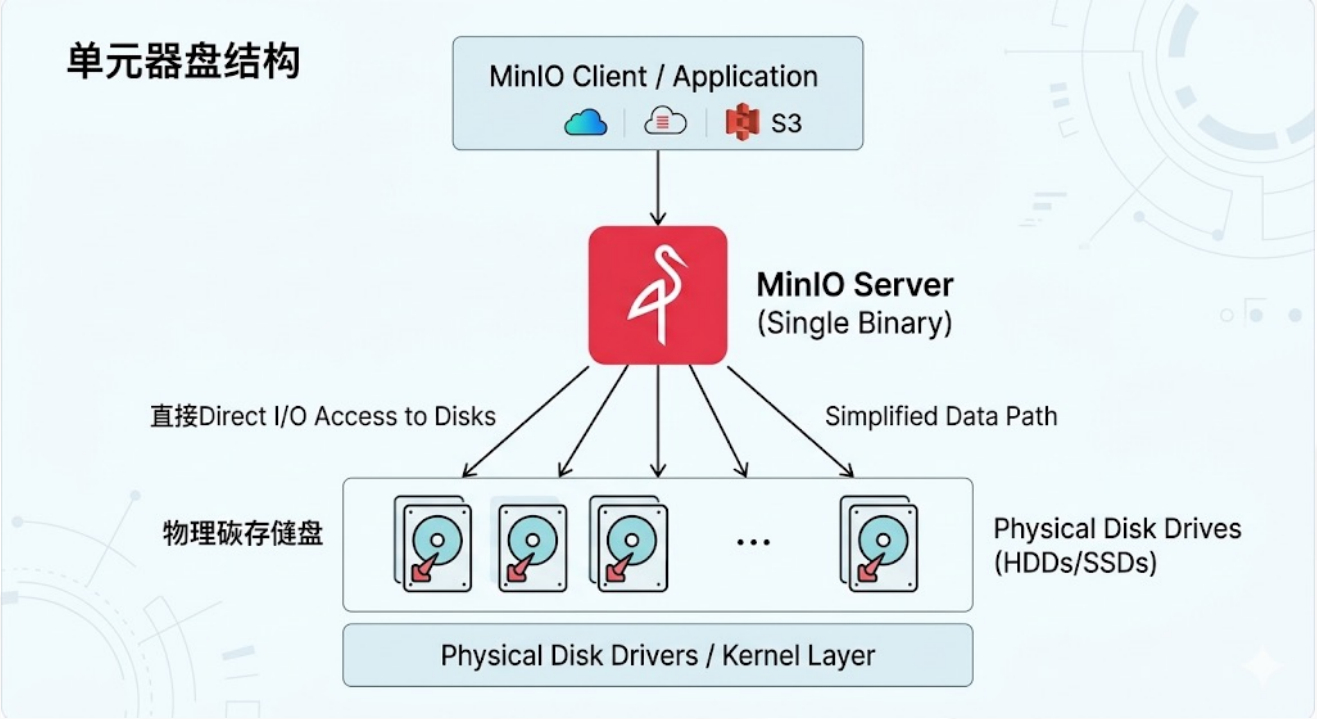
原生纠删码保护：即使单节点运行，也强制使用纠删码（Erasure Coding）加位操作，防静默数据损坏（Bit Rot）。

❌ **传统对象存储**
复杂依赖（DB / 缓存）
运维复杂
写入吞吐有限

✅ **MinIO**
单二进制零依赖
原生纠删码
GB/s 吞吐

04 MinIO 部署哲学

不提倡几千个节点组成一个超级大集群（元数据易瓶颈），而是提倡将大型系统划分为多个“适中规模的Tenant（租户集群）”，通过Kubernetes统一调度管理。



05 对象存储 API 调用逻辑 (1/2)

对象存储没有传统意义的 mount和 open()，一切操作基于标准的HTTP动词。

核心代码片段：使用boto3连接并上传数据

```
import boto3


# 1. 客户端初始化：指定S3兼容端点与凭证
s3 = boto3.client('s3',
    endpoint_url='http://localhost:9000',
    aws_access_key_id='admin',
    aws_secret_access_key='password123'
)
```

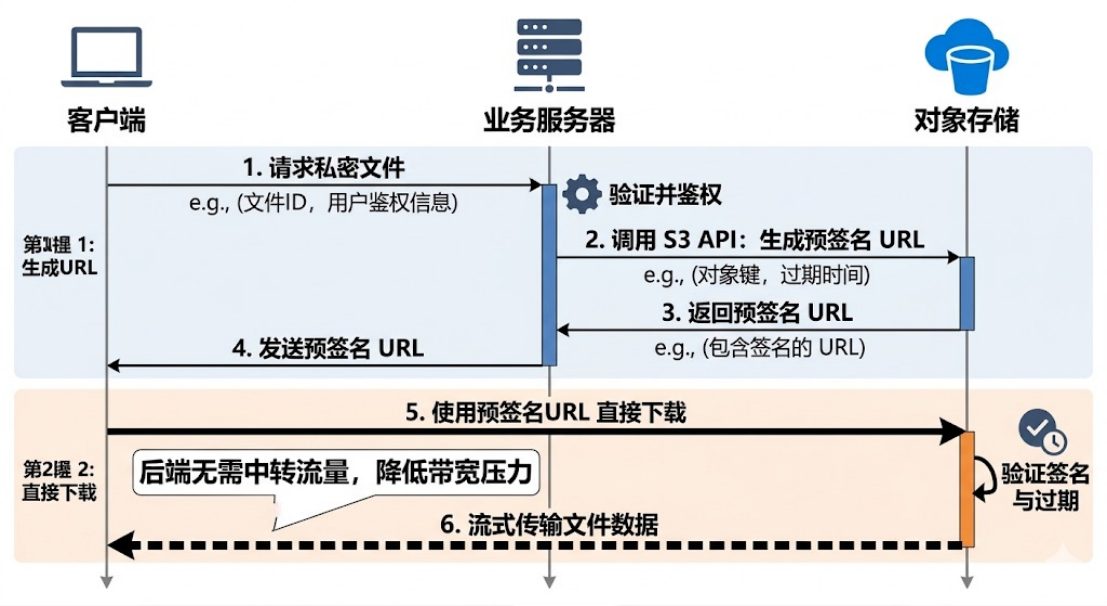
05 对象存储 API 调用逻辑 (2/2)

```
# 2. 将内存中的JSON数据直接作为Object推送到Bucket
s3.put_object(
    Bucket='demo-bucket',
    Key='config/app.json',
    Body=b'{"version": "1.0"}',
    ContentType='application/json',
    Metadata={'author': 'dai-penglin'}
# 自定义元数据绑定
)
```

```
# 3. 生成预签名URL (Presigned URL)
url = s3.generate_presigned_url('get_object',
    Params={'Bucket': 'demo-bucket', 'Key': 'config/app.json'},
    ExpiresIn=3600)  # 授予1小时的临时下载权限
```

06 预签名URL

预签名URL是对象存储最具魅力的功能之一。当企业需要向外部用户提供带时效性的私密文件下载（如网盘分享链接、付费电子书下载）时，后端无需中转流量，只需签发一串带有时效签名的S3直链，用户直接从云端下载，极大降低了应用服务器带宽压力。



5 Ceph统一存储

- Ceph 的诞生
- RADOS / CRUSH 算法
- 三大存储接口

01 传统存储痛点与Ceph的诞生

西南交通大学 Southwest Jiaotong University

在过去，企业建设私有云需要采购并维护三套完全不同的系统：

- 跑数据库：买昂贵的FC SAN（提供块存储）
- 共享文件：搭一堆NFS或GlusterFS服务器（提供文件存储）
- 存海量图片：部署一套MinIO或买商用S3（提供对象存储）

痛点：容量无法互通，运维极其痛苦，硬件资源严重孤岛化。

FC SAN
跑数据库
昂贵专用硬件
孤立扩容难

NFS / GlusterFS
共享文件
多机挂载
性能上限低

对象存储
海量图片视频
成本敏感
另一套硬件

01 Ceph：统一存储架构

Ceph 的核心愿景就是“All in One”。它底层构建了一个无限扩展的可靠分布式对象存储引擎（RADOS），然后在其上层“模拟”出块、文件、对象三种标准接口。一套硬件，满足所有需求。



02 Ceph三大核心守护进程

OSD (Object Storage Daemon)：真正的干活者，一块硬盘对应一个OSD进程，负责数据的存储、复制、恢复、心跳。

MON (Monitor)：集群的“最高法院”，维护着整个集群拓扑图状态机（OSD Map等）。必须奇数部署（基于Paxos保证强一致）。

MDS (Metadata Server)：仅在提供CephFS文件存储时才需要，专门负责处理和缓存目录树元数据。







03 RADOS（可靠自主分布式对象存储）

RADOS层是完全无单点的自动修复网络。在RADOS看来，所有上层的数据（不管是100GB的块设备，还是一个多媒体文件），全都被无情地切分成默认**4MB**大小的RADOS Object。

上层：虚拟机磁盘镜像 my_vm_disk (100GB)

切片：被Librados客户端切分为 25,600 个 4MB 的RADOS Object

底座：均匀打散存入 RADOS 底层的千百块物理磁盘（OSD）中

100 GB ÷ 4 MB = 25,600 个 RADOS Object

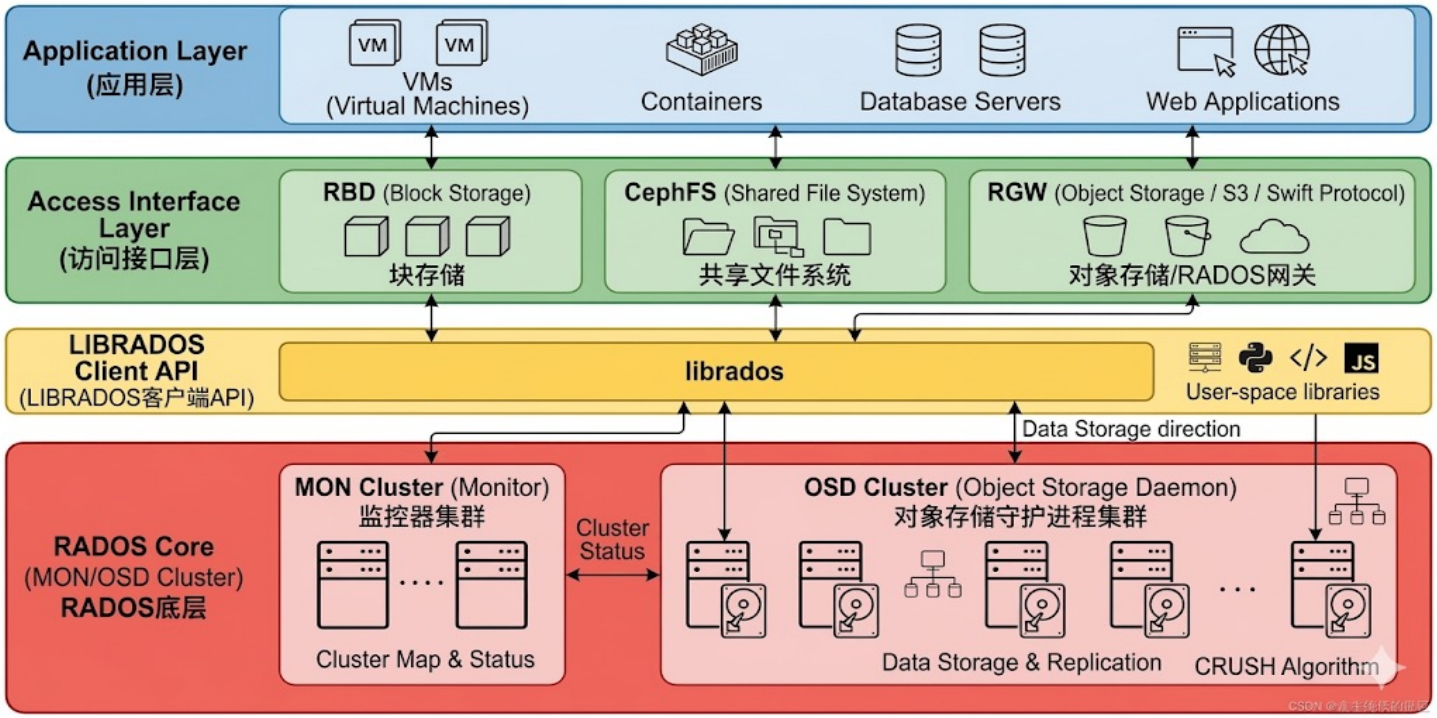
上层任意大小数据 → 切片均匀打散到全集群千百块磁盘

03 为什么切分成4MB小块？

西南交通大学 Southwest Jiaotong University

极致并发：100GB文件的读写被分散到全集群上百块磁盘同时进行，避免了单点热点。

极速恢复：若某块硬盘损坏，不是一台备机来恢复，而是全集群所有硬盘共同参与“碎片重组”，几百TB数据可在数十分钟内修复完毕。



04 CRUSH算法

HDFS需要NameNode的内存记录“每一个Block存在哪台机器”。当数据达到EB级（亿万个对象），任何中心节点的内存都会被撑爆，且每次寻址的网络开销极大。

CRUSH（Controlled Replication Under Scalable Hashing）算法是Ceph的核心专利。它让客户端不查任何表，仅靠本地计算就能算出数据该存到哪块硬盘。

客户端寻址核心逻辑（伪代码）：

```
1. 哈希:  PG_ID = Hash(Object_Name) % PG_Total_Num
2. CRUSH:  OSD_List = CRUSH(PG_ID, Cluster_Map, Rule_Set)
3. 直连:  Client 直接连接 OSD_List[0] 写入数据
```

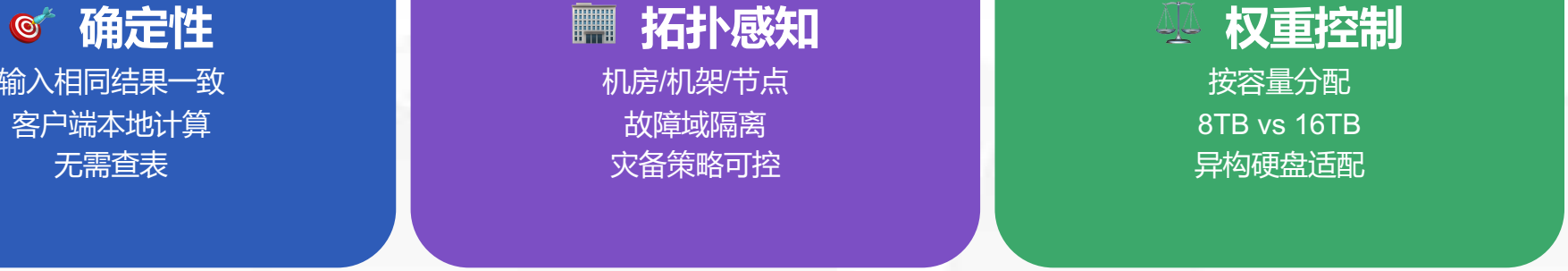
04 CRUSH的智能特性

西南交通大学 Southwest Jiaotong University

伪随机与确定性：同样的输入，任何客户端在任何时候计算出的磁盘分布绝对一致。

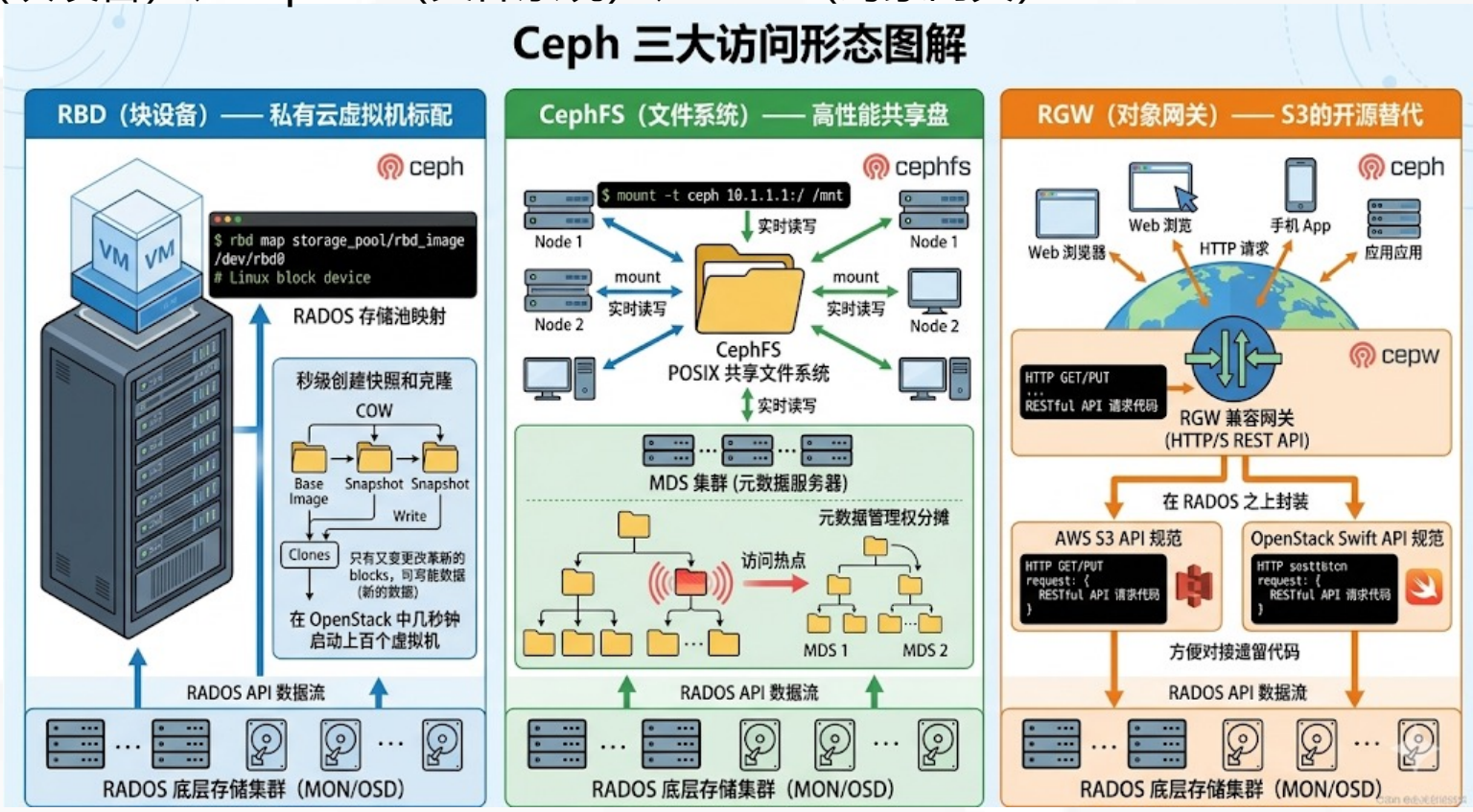
故障域隔离感知（Topology Aware）：配合CRUSH Rule，可以强制要求“3个副本必须落在不同的机架/机房/甚至是不同城市的机房”，实现物理级别的灾备。

权重控制：8TB的硬盘和16TB的硬盘，分配数据的概率完全可以随容量大小按比例倾斜。



05 Ceph 三大存储接口 (1/3)

RBD（块设备）、CephFS（文件系统）、RGW（对象网关）



05 Ceph 三大存储接口 (2/3)：RBD / CephFS

西南交通大学 Southwest Jiaotong University

RBD（块设备）—— 私有云虚拟机标配

原理：将RADOS存储池映射成Linux下的块设备（如 /dev/rbd0）。

杀手锏：支持秒级创建快照（Snapshot）和克隆（Clone）。在OpenStack中，用RBD启动上百个虚拟机只需几秒钟（因为利用了写时复制COW机制）。

CephFS（文件系统）—— 高性能共享盘

原理：提供高度兼容POSIX的文件系统，支持多台机器同时mount并实时读写。

杀手锏：MDS集群采用动态子树分区策略。如果某个目录下文件访问量激增（热点），Ceph会自动把该目录的元数据管理权分摊给其他MDS节点。

05 Ceph 三大存储接口 (3/3)：RGW

西南交通大学 Southwest Jiaotong University

RGW（对象网关）—— S3的开源替代

原理：在RADOS之上封装的一层HTTP网关转换协议。

杀手锏：原生支持AWS S3和OpenStack Swift双重API规范，方便企业内不同时期遗留代码的对接。

AWS S3 API

主流 SDK 直接对接

boto3 / aws-sdk

迁移零成本

OpenStack Swift API

历史代码兼容

OpenStack 生态

无需重写

06 现代云基础设施的底座

西南交通大学 Southwest Jiaotong University

黄金搭档 1：OpenStack + Ceph

这是构建传统IaaS私有云的标准范式。

Cinder组件（云硬盘）无缝对接 Ceph RBD

Glance组件（镜像服务）将系统镜像存为 RBD

Manila组件（共享文件）对接 CephFS

黄金搭档 2：Kubernetes + Ceph (Rook)

云原生时代的持久化存储解法。

利用官方的 ceph-csi插件，K8s通过创建 PVC (PersistentVolumeClaim)即可自动触发Ceph动态分配存储。

Rook项目：将Ceph本身的复杂运维操作（扩容、恢复）全部封装为K8s Operator，把Ceph变成了K8s上的原生一等公民。

6 分布式文件系统对比

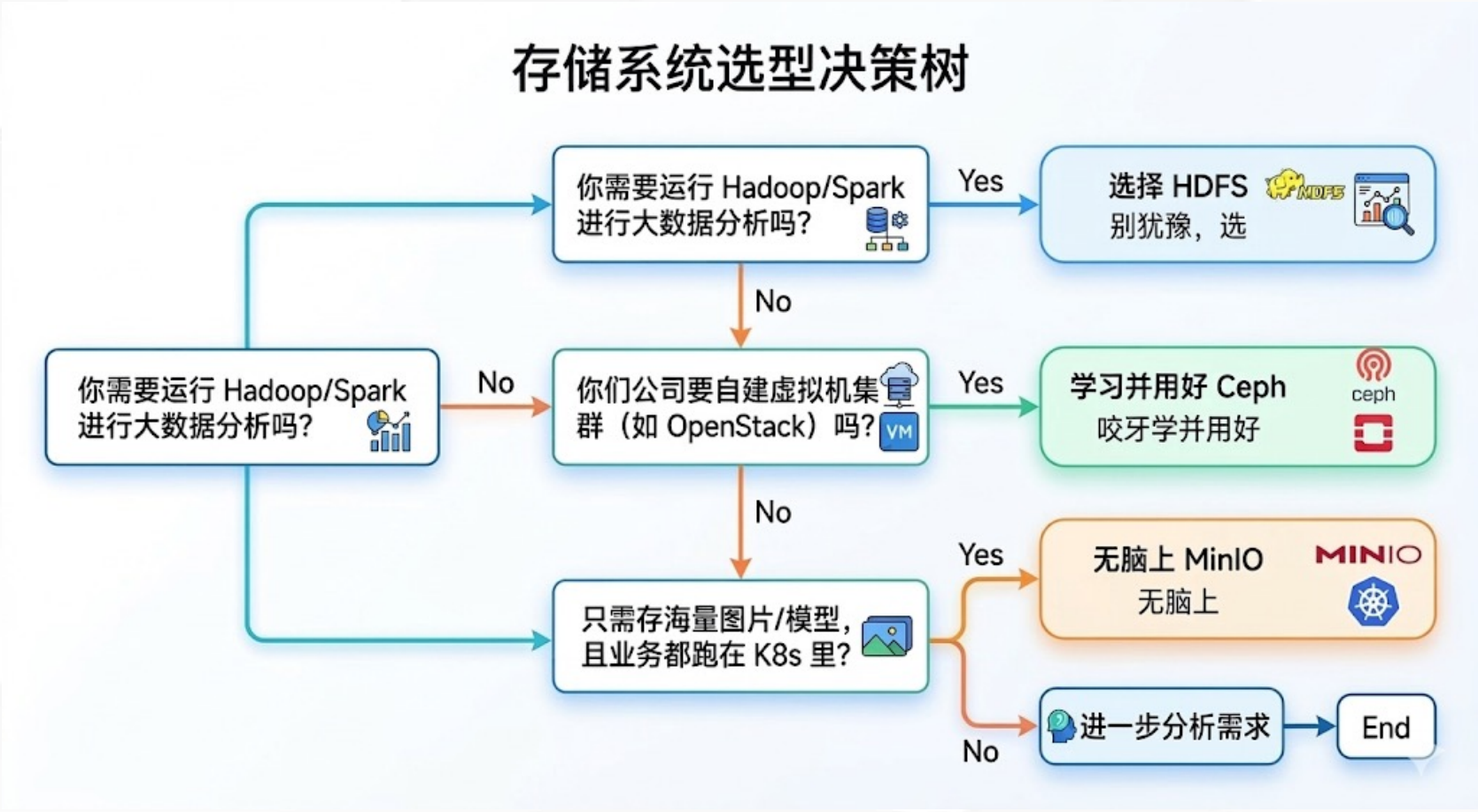
- 如何选型
- 云厂商存储矩阵
- 云端计费陷阱

01 分布式存储全景对比

西南交通大学 Southwest Jiaotong University

对比维度	HDFS	Ceph	MinIO	GlusterFS
架构流派	中心化主从架构	去中心化统一架构	轻量级对象架构	无中心哈希架构
元数据管理	纯内存 NameNode	算法计算 (CRUSH)	分布式去中心化	弹性哈希算法
核心优势	计算向数据移动	块 / 文件 / 对象通吃	极致轻量、超高吞吐	部署极其简单
痛点 / 劣势	极度害怕海量小文件	学习曲线陡、运维灾难	仅支持对象接口	小文件性能严重下滑
S3 生态兼容	不支持	较好（RGW）	完美原生兼容	不支持
适用黄金场景	大数据批处理 (Spark 等)	IaaS 私有云底座 (OpenStack)	云原生 AI 数据湖	传统 NAS 平替

02 CIO选型决策树


存储系统选型决策树
你需要运行 Hadoop/Spark 进行大数据分析吗?
Yes
选择 HDFS
别犹豫，选
No
你需要运行 Hadoop/Spark 进行大数据分析吗?
No
你们公司要自建虚拟机集群（如 OpenStack）吗?
Yes
学习并用好 Ceph
咬牙学并用好
ceph
No
只需存海量图片/模型，且业务都跑在 K8s 里?
Yes
无脑上 MinIO
无脑上
MINIO
No
进一步分析需求
End

03 云厂商的存储矩阵

对绝大多数创业公司而言，不再自建物理存储，而是采购公有云服务。各厂商也严格遵循三大存储类型的矩阵分布。

存储类型	AWS (亚马逊)	Azure (微软)	Aliyun (阿里云)	Tencent (腾讯云)
块存储	EBS	Managed Disks	云盘 / ESSD	CBS
文件存储	EFS / FSx	Azure Files	NAS	CFS
对象存储	S3	Blob Storage	OSS	COS
极冷归档	S3 Glacier	Archive Tier	归档存储	归档存储

03 云端计费与选型隐藏陷阱

西南交通大学 Southwest Jiaotong University

并非越冷越划算：归档存储虽然每月每GB极度便宜，但如果你的应用频繁调用/读取冷数据，其产生的“数据取回费”和“API请求费”可能远高于标准层存储！

跨区流量陷阱：不同可用区（AZ）甚至内网之间传输海量数据，经常会产生高昂的跨区流量费用。

10×
热数据放冷层
月费可暴涨 10 倍

$0.09/GB
跨可用区流量
海量传输痛点

8 思考题、课后作业与实践

- 深度思考题
- 课后作业
- Q&A

01 深度思考题

西南交通大学 Southwest Jiaotong University

Q1 20 亿张 50KB 头像存入 HDFS 会发生什么灾难？作为架构师该如何选型？

Q2 GFS 放弃严格一致性，为何 Google 敢这么做？上层应用如何配合？

Q3 热门短视频存入 S3 Glacier 后账单暴涨 10 倍，为什么？

02 本讲知识要点全局回顾

西南交通大学 Southwest Jiaotong University

核心领域	关键知识点与底层哲学
存储形态演进	从块（裸盘）→ 文件（目录树）→ 对象（扁平 KV）
GFS 的时代启发	承认硬件易碎，用软件容灾；控制流与数据流彻底分离
HDFS 核心机制	NameNode (内存瓶颈) / DataNode；机架感知；大区块设计
HDFS 进阶演进	HA 高可用防脑裂 / Federation 解耦内存 / EC 纠删码降本
对象存储 S3 模型	抛弃目录树换取无限横向扩展；数据生命周期智能分层
MinIO 架构特色	极简二进制、原生纠删码、Kubernetes 时代的云原生标配
Ceph 统一存储	一套底座支持三大接口；CRUSH 算法（用算力取代查表）

03 下次课预告 & Q&A

西南交通大学 Southwest Jiaotong University

下次课预告：第9讲 并行编程模型与计算框架

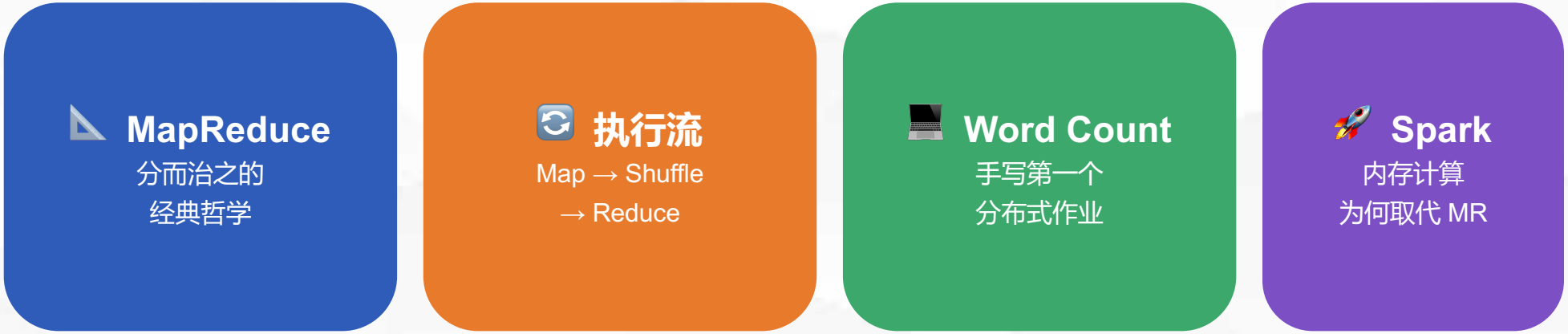
存储解决了“数据放哪里”的问题，下一步是“数据怎么算”！

MapReduce 编程模型：分而治之的经典哲学

核心执行流：Map（映射） ➔ Shuffle（洗牌） ➔ Reduce（归约）

Hadoop MapReduce 实践：带你手写第一个分布式 Word Count

时代演进：为什么 Spark 的内存计算最终取代了 MapReduce?



04 课后作业与实践（选做）

西南交通大学 Southwest Jiaotong University

基础实战：使用 Docker 5分钟拉起单节点 MinIO，利用 Python boto3生成一个有效期为 5 分钟的私密下载链接，并尝试访问。

深度调研：查阅资料对比 一致性哈希 (Consistent Hashing) 与 Ceph CRUSH算法的区别，写出不少于 300 字的技术分析。

Lab 1：MinIO 实战

⭐⭐ Docker + boto3

🎯 预签名 URL 5 分钟有效

Lab 2：算法对比

⭐⭐⭐ 一致性哈希 vs CRUSH

🎯 不少于 300 字技术分析